TP7 - Integración y E2E de PhotoMatch

1. Objetivo y teoría

La unidad de release es una imagen inmutable: build once, deploy many. Integración comprueba API y PostgreSQL sin navegador. E2E usa un navegador real y recorre acciones de usuario. El gate convierte sus resultados en condición para producción.

2. Situación inicial

PhotoMatch tenía EJS y formularios, pero no una API JSON de clientes ni Playwright. El smoke sólo decía que **/health** respondía. El PR #18 reconstruía imágenes por componente y no tenía pruebas contra un entorno desplegado.

3. Archivos modificados

backend/src/controllers/apiClientController.js, **backend/src/routes/index.js**, **backend/src/app.js**, **backend/src/middleware/auth.js**, **frontend/e2e/api.spec.js**, **frontend/e2e/app.spec.js**, **frontend/playwright.config.js**, **frontend/package*.json** y **.github/workflows/ci.yml**. También se corrigieron controlador y formulario EJS de clientes.

4. Código importante

La API **/api/clients** usa el repositorio real y la misma **validateClient** de la interfaz. La ruta exige sesión. Responde 201/400/404/204 según operación. La suite de integración hace alta-GET-DELETE-GET, comprueba 400 sin inserción y edita un cliente; usa nombres únicos y limpia. La suite E2E llena formularios en Chromium, verifica el dato propio y lo borra; el error de nombre vacío no persiste. El workflow encadena **deploy-qa** → **integracion** → **e2e** → **deploy-prod**, con reportes separados incluso cuando falla una suite.

5. Decisiones y alternativas

Se agregó una API pequeña para clientes porque la consigna pide el fixture **request** contra la API real; no se creó una base simulada. Las pruebas de navegador usan el front Nginx, no un servidor de test. **workers: 1** evita conflictos entre suites que comparten QA; **retries: 1** tolera transitorios sin ocultar fallos persistentes. PROD recibe exactamente los tags verificados en QA.

6. Herramientas

Playwright **request** mantiene cookies de sesión y hace HTTP sin browser. Playwright Chromium ejerce UI, conserva screenshot/trace al fallar y genera HTML. **upload-artifact** guarda reportes; **needs** bloquea promoción si falla integración o E2E.

7. Tests y evidencia real

Con PostgreSQL y Nginx locales: integración 3/3 y E2E 3/3, cero saltados. El navegador detectó un 500 real en **/clients/new**; tras corregirlo, todas pasaron. **Faltan** corridas sobre QA desplegado por **main**, rojo E2E provocado por un cambio de aplicación, fix, gate de PROD y **v7.0.0**.

8. Problemas y solución

EJS interpretó el local **client** como opción de compilación y **include** dejó de ser función. Se renombró el local del formulario a **customer**. Se repitieron las tres E2E para confirmar el arreglo.

9. Preguntas del profesor

¿Por qué no alcanza el smoke? Sólo consulta salud y SHA; un formulario puede devolver 500. **¿Qué diagnostica verde integración y rojo E2E?** API/DB funcionaron, pero una ruta o interacción de UI falló. **¿Por qué limpiar datos?** QA se comparte entre corridas; datos viejos falsean aserciones. **¿Qué pasa si E2E falla? deploy-prod** no llega al gate humano.

10. Demo de defensa

Mostrar los specs, los reportes de una corrida verde y una roja genuina, el diff que rompió la app, y el job PROD omitido. Después mostrar fix, reportes verdes, mismo SHA en QA/PROD y release apuntando al commit desplegado. Estas evidencias de GitHub aún deben generarse.